22. Katmanlı Mimari & Separation of Concerns: Sorumlulukları Ayırmak

Controller, Service, Repository Katmanlarını Birbirine Karıştırmama Kılavuzu
Bölüm 4 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Bir API Controller'ının içine SQL sorgusu yazmak neden yazılım dünyasının en büyük günahlarından biridir ve katmanlı mimari sistemi nasıl korur?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Separation of Concerns (İlgi Alanlarının Ayrımı), sistemin her katmanının sadece kendi işini bilmesidir. - 📌 Controller sadece HTTP protokolünü (istek/yanıt) yönetir; Service saf iş mantığını yürütür; Repository ise veriye erişir. - 📌 Yapay zekaya bu katmanları açıkça belirttiğinizde, AI çok daha temiz, modüler ve kurumsal kod üretir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Lüks bir oteli hayal edin.

Otelde 3 ana görevli var: 1. **Resepsiyonist (Controller):** Kapıda müşteriyi karşılar, kimliğini kontrol eder ve isteğini alır. Asla mutfağa girip yemek pişirmez! 2. **Şef Aşçı (Service):** Siparişin malzemelerini hazırlar, tarife göre pişirir. Müşterinin kredi kartıyla ilgilenmez! 3. **Kiler Sorumlusu (Repository):** Domatesin, etin hangi rafta olduğunu bilir; şef isteyince raftan çıkarıp verir.

Eğer resepsiyonist masayı bırakıp mutfakta omlet yapmaya, kilerdeki domatesleri saymaya kalkarsa otelde kaos çıkar!

İşte Controller içine veritabanı sorgusu yazmak, o resepsiyonistin mutfakta tencere yıkamasıdır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

3 Katmanlı Mimari Akışı:

```
[İstemci (React/Mobil)] ──(HTTP İsteği)──> [1. CONTROLLER KATMANI]
                                                    |
                                             (İş Talebi)
                                                    ▼
                                            [2. SERVICE KATMANI]
                                             (İş Mantığı & Kurallar)
                                                    |
                                             (Veri Talebi)
                                                    ▼
                                            [3. REPOSITORY KATMANI]
                                                    |
                                            (SQL / ORM Sorgusu)
                                                    ▼
                                            [VERİTABANI (Postgres/Mongo)]
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Temiz Mimari (Clean Architecture) Uzmanı
Node.js / Express ve Prisma kullanan bir kullanıcı oluşturma özelliğimiz var.

## GÖREV:
Bu özelliği 3 katmanlı mimariye uygun şekilde 3 ayrı dosyada tasarla:
1. `user.controller.ts`: Sadece HTTP req/res, durum kodları ve Zod doğrulaması ile ilgilensin.
2. `user.service.ts`: Şifre hashleme ve iş kuralları ile ilgilensin (asla HTTP nesnesi almasın!).
3. `user.repository.ts`: Sadece Prisma sorgusu ile veritabanına kayıt yapsın.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Felaket: Controller içinde hem validasyon, hem hashleme, hem SQL!
app.post("/users", async (req, res) => {
  if (!req.body.email) return res.status(400).send("Email gerekli");
  const hashed = await bcrypt.hash(req.body.password, 10);
  const user = await prisma.user.create({ data: { email: req.body.email, password:
hashed } });
  await sendEmail(user.email);
  res.json(user); // Şifre hash'i bile frontend'e sızdı!
});
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
// 1. REPOSITORY KATMANI (user.repository.ts)
export class UserRepository {
  async create(data: { email: string; passwordHash: string }) {
    return await prisma.user.create({ data });
  }
  async findByEmail(email: string) {
    return await prisma.user.findUnique({ where: { email } });
  }
}

// 2. SERVICE KATMANI (user.service.ts - Saf İş Mantığı)
export class UserService {
  constructor(private readonly userRepo: UserRepository) {}

  async registerUser(dto: RegisterUserDto): Promise<SafeUser> {
    const existing = await this.userRepo.findByEmail(dto.email);
    if (existing) throw new ConflictError("E-posta zaten kayıtlı.");

    const passwordHash = await hashPassword(dto.password);
    const user = await this.userRepo.create({ email: dto.email, passwordHash });

    return { id: user.id, email: user.email }; // Şifre asla dönmez!
  }
}

// 3. CONTROLLER KATMANI (user.controller.ts - Sadece HTTP Protokolü)
export class UserController {
  constructor(private readonly userService: UserService) {}

  async handleRegister(req: Request, res: Response): Promise<void> {
    const validated = RegisterUserSchema.parse(req.body);
    const user = await this.userService.registerUser(validated);
    res.status(201).json({ success: true, data: user });
  }
}
```

💡 **Kod Analizi:** Katmanlar ayrıldığında; veritabanını Prisma'dan TypeORM'a geçirmek sadece Repository'yi etkiler. Web framework'ünü Express'ten Fastify'a geçirmek ise sadece Controller'ı etkiler. Service katmanındaki iş mantığına asla dokunulmaz!

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Service fonksiyonlarına Express `req` ve `res` nesnelerini parametre olarak geçmek.
  - **Çözüm:** Service katmanı HTTP'den habersiz olmalıdır; sadece saf JavaScript objeleri (DTO) almalıdır.
- **Tuzak:** Controller'ı atlayıp doğrudan veritabanı modelini frontend'e döndürerek şifre gibi gizli alanları sızdırmak.
  - **Çözüm:** Daima Response DTO kullanarak sadece güvenli alanları dönün.
- **Tuzak:** Basit CRUD işlemlerinde gereksiz 10 katman oluşturup aşırı karmaşıklık yaratmak.
  - **Çözüm:** İş mantığı olmayan yerlerde katmanları gereksiz derinleştirmeyin; pragmatik olun.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 22: Katmanlı Mimari - Controller, Service ve Repository*

- 🖥️ **Slayt 1:** Separation of Concerns (İlgi Alanlarının Ayrımı) nedir?
- 🖥️ **Slayt 2:** 3 Katmanın Görev Dağılımı: Resepsiyonist, Şef ve Kiler.
- 🖥️ **Slayt 3:** Service katmanını HTTP bağımsız (framework-agnostic) tutmanın gücü.
- 🖥️ **Slayt 4:** Canlı Demo: Spagetti Express rotasını 3 katmana refactor etmek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Tek dosyalık bir `app.post('/register')` kodunu açın. Adım adım `repository`, `service` ve `controller` dosyalarına bölün. Service katmanı için sahte bir repository ile 1 dakikada unit test yazarak çalıştığını gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Mevcut projenizde Controller içinde doğrudan veritabanı sorgusu yapılan bir fonksiyonu bulun ve veritabanı kodunu bir Repository fonksiyonuna taşıyın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: 3 Katmanlı Mimaride 'Service' katmanının temel sorumluluğu nedir? - A) HTTP durum kodlarını (200, 404) belirlemek - B) Şirketin saf iş kurallarını (business logic) ve hesaplamalarını yürütmek - C) Doğrudan SQL sorgusu yazmak - D) CSS butonlarını renklendirmek > **Doğru Yanıt:** B > **Açıklama:** Service katmanı iş kurallarının kalbidir; ne HTTP protokolünden ne de veritabanının türünden etkilenmeden mantığı işletir.

Soru 2: Service fonksiyonuna `(req, res)` nesnelerini parametre olarak geçmek neden kötü bir mimari alışkanlıktır? - A) Fonksiyonu yavaşlatması - B) Service katmanını Express'e ve HTTP protokolüne sıkı sıkıya bağlayarak CLI, kuyruk (queue) veya test ortamlarında yeniden kullanılmasını imkansız kılması - C) TypeScript'in hata vermesi - D) Monitörün kapanması > **Doğru Yanıt:** B > **Açıklama:** Service saf verilerle (DTO) çalışmalıdır; böylece aynı servis hem bir HTTP isteğinden hem de bir cron job veya kuyruk mesajından çağrılabilir.